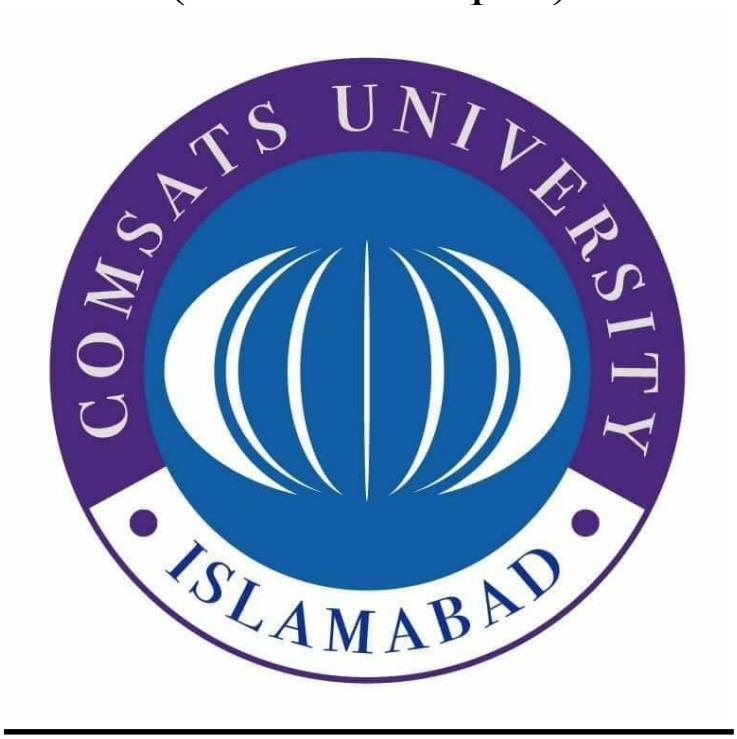
Comsats University Islamabad

(Attock Campus)



Project Report

Project Tittle: Railway Management System

Name	Reg.No
Manahil Kamran	Fa24-bse-025
Syeda Aliza Hashmi	Fa24-bse-030
Zineera Zainab Qureshi	Fa24-bse-033
Umer Farooq	Fa24-bse-054

Submitted to: Mam Munazza

Subject: Software Quality Engineering

Date: 18 May 2026

Table Of Contents

1. Project Objective

- Develop a functional Python system.
- Apply manual + automated testing.
- Identify and report bugs.
- Perform code quality and security analysis.
- Use GitHub for version control.

2. System Development

System chosen:

Railway Management System

Core Functionalities:

1. Train Module:

- **Add Train:** Add new trains with ID, name, route, and seat capacity.
- **Update Train**: Modify train details (e.g., available seats).
- **Delete Train:** Remove a train record.
- **View Schedule:** Display all trains with their details.
- **Search by Route:** Find trains running on a specific route.

2. Passenger Module:

- **Add Passenger:** Register new passengers with ID, name, and contact info.
- **Update Passenger:** Modify passenger contact details.
- **Delete Passenger:** Remove passenger records.
- **View Passengers:** Display all registered passengers.
- **Search Passenger:** Find a passenger by ID.

3. Booking Module:

- **Book Ticket:** Reserve a seat for a passenger on a train (reduces seat count).
- **Cancel Ticket:** Cancel a booking (restores seat count).
- **View Bookings:** Display all bookings with passenger and train IDs.
- **Search Booking:** Find bookings by passenger ID.
- **Check Availability:** Show remaining seats for a train.

Screenshot of code:

```
class Train:  1 usage
    def __init__(self, train_id, name, route, seats):
        self.train_id = train_id
        self.name = name
        self.route = route
        self.seats = seats

class TrainModule:  2+ usages
    def __init__(self):
        self.trains = {}

    def add_train(self, train_id, name, route, seats):  2+ usages
        self.trains[train_id] = Train(train_id, name, route, seats)

    def update_train(self, train_id, seats):  1 usage
        if train_id in self.trains:
            self.trains[train_id].seats = seats

    def delete_train(self, train_id):  1 usage
        if train_id in self.trains:
            del self.trains[train_id]

    def view_schedule(self):  1 usage
        return [(t.train_id, t.name, t.route, t.seats) for t in self.trains.values()]

    def search_by_route(self, route):  1 usage
        return [t for t in self.trains.values() if t.route == route]


class Passenger:  1 usage
    def __init__(self, passenger_id, name, contact):
        self.passenger_id = passenger_id
        self.name = name
        self.contact = contact

class PassengerModule:  2+ usages
    def __init__(self):
        self.passengers = {}

    def add_passenger(self, passenger_id, name, contact):  2+ usages
        self.passengers[passenger_id] = Passenger(passenger_id, name, contact)

    def update_passenger(self, passenger_id, contact):  1 usage
        if passenger_id in self.passengers:
            self.passengers[passenger_id].contact = contact

    def delete_passenger(self, passenger_id):  1 usage
        if passenger_id in self.passengers:
            del self.passengers[passenger_id]

    def view_passengers(self):  1 usage
        return [(p.passenger_id, p.name, p.contact) for p in self.passengers.values()]

    def search_passenger(self, passenger_id):  1 usage
        return self.passengers.get(passenger_id,  default: None)
```

```python
class Booking:  1 usage
    def __init__(self, booking_id, passenger_id, train_id):
        self.booking_id = booking_id
        self.passenger_id = passenger_id
        self.train_id = train_id

class BookingModule:  2+ usages
    def __init__(self, train_module):
        self.bookings = {}
        self.train_module = train_module

    def book_ticket(self, booking_id, passenger_id, train_id):  7+ usages
        train = self.train_module.trains.get(train_id)
        if train and train.seats > 0:
            train.seats -= 1
            self.bookings[booking_id] = Booking(booking_id, passenger_id, train_id)
            return "Ticket booked successfully!"
        return "Booking failed. Train not found or no seats."
    return "Booking failed. Train not found or no seats."

def cancel_ticket(self, booking_id):  1 usage
    if booking_id in self.bookings:
        train_id = self.bookings[booking_id].train_id
        self.train_module.trains[train_id].seats += 1
        del self.bookings[booking_id]
        return "Ticket cancelled successfully!"
    return "Cancellation failed."

def view_bookings(self):  1 usage
    return [(b.booking_id, b.passenger_id, b.train_id) for b in self.bookings.values()]

def search_booking(self, passenger_id):  1 usage
    return [b for b in self.bookings.values() if b.passenger_id == passenger_id]

def check_availability(self, train_id):  1 usage
    train = self.train_module.trains.get(train_id)
    return train.seats if train else "Train not found"
```

system is executable:

```
C:\Users\manah\PyCharmMiscProject\projects\.venv\Scripts\python.exe C:\Users\manah\PyCharmMiscProject\projects\SQE\project.py

Process finished with exit code 0
```

3. Manual Testing

Document form:

Test ID	Module	Functionality	Input	Expected Output	Actual Output	Status
TC01	TrainModule	Add Train	Train ID=102, Name="FastTrack", Route="Rawalpindi-Lahore", Seats=50	Train added successfully (ID=102 exists in trains)	Train added successfully	Pass
TC02	TrainModule	Update Train	Train ID=101, Seats=10	Seats updated to 10	Seats updated to 10	Pass
TC03	TrainModule	Delete Train	Train ID=101	Train removed from records	Train removed	Pass
TC04	TrainModule	View Schedule	Call view_schedule()	Returns list of trains	Returns list of trains	Pass
TC05	TrainModule	Search by Route	Route="Lahore-Karachi"	Returns train "Express"	Returns train "Express"	Pass
TC06	PassengerModule	Add Passenger	Passenger ID=2, Name="Ali", Contact="0300-7654321"	Passenger added successfully	Passenger added successfully	Pass
TC07	PassengerModule	Update Passenger	Passenger ID=1, Contact="0300-9999999"	Contact updated	Contact updated	Pass
TC08	PassengerModule	Delete Passenger	Passenger ID=1	Passenger removed	Passenger removed	Pass
TC09	PassengerModule	View Passengers	Call view_passengers()	Returns list of passengers	Returns list of passengers	Pass
TC10	PassengerModule	Search Passenger	Passenger ID=1	Returns passenger "Manahil"	Returns passenger "Manahil"	Pass
TC11	BookingModule	Book Ticket (Success)	Booking ID=201, Passenger ID=1, Train ID=101	"Ticket booked successfully!" and seats reduced	"Ticket booked successfully!" and seats reduced	Pass
TC12	BookingModule	Book Ticket (Failure)	Book 3 tickets on Train ID=101 (only 2 seats)	"Booking failed. Train not found or no seats."	"Booking failed. Train not found or no seats."	Pass
TC13	BookingModule	Cancel Ticket	Booking ID=201	"Ticket cancelled successfully!" and seats restored	"Ticket cancelled successfully!" and seats restored	Pass
TC14	BookingModule	View Bookings	Call view_bookings()	Returns list of bookings	Returns list of bookings	Pass
TC15	BookingModule	Search Booking	Passenger ID=1	Returns booking with Passenger ID=1	Returns booking with Passenger ID=1	Pass
TC16	BookingModule	Check Availability	Train ID=101	Returns integer seat count	Returns integer seat count	Pass

Unit-Testing:

Screenshot of code:

```python
# testing.py
import unittest
from project import TrainModule, PassengerModule, BookingModule


class TestRailwaySystem(unittest.TestCase):

    def setUp(self):
        # Initialize modules
        self.train_module = TrainModule()
        self.passenger_module = PassengerModule()
        self.booking_module = BookingModule(self.train_module)

        # Add sample train and passenger
        self.train_module.add_train( train_id: 101,  name: "Express",  route: "Lahore-Karachi",  seats: 2)
        self.passenger_module.add_passenger( passenger_id: 1,  name: "Manahil",  contact: "0300-1234567")

    # ---------- Train Module Tests ----------
    def test_add_train(self):
        self.train_module.add_train( train_id: 102,  name: "FastTrack",  route: "Rawalpindi-Lahore",  seats: 50)
        self.assertIn( member: 102, self.train_module.trains)

    def test_update_train(self):
        self.train_module.update_train( train_id: 101,  seats: 10)
        self.assertEqual(self.train_module.trains[101].seats,  second: 10)

    def test_delete_train(self):
        self.train_module.delete_train(101)
        self.assertNotIn( member: 101, self.train_module.trains)
```

```python
    def test_view_schedule(self):
        schedule = self.train_module.view_schedule()
        self.assertTrue(len(schedule) > 0)

    def test_search_by_route(self):
        result = self.train_module.search_by_route("Lahore-Karachi")
        self.assertEqual(result[0].name,  second: "Express")

    # ---------- Passenger Module Tests ----------
    def test_add_passenger(self):
        self.passenger_module.add_passenger( passenger_id: 2,  name: "Ali",  contact: "0300-7654321")
        self.assertIn( member: 2, self.passenger_module.passengers)

    def test_update_passenger(self):
        self.passenger_module.update_passenger( passenger_id: 1,  contact: "0300-9999999")
        self.assertEqual(self.passenger_module.passengers[1].contact,  second: "0300-9999999")

    def test_delete_passenger(self):
        self.passenger_module.delete_passenger(1)
        self.assertNotIn( member: 1, self.passenger_module.passengers)

    def test_view_passengers(self):
        passengers = self.passenger_module.view_passengers()
        self.assertTrue(len(passengers) > 0)
```

```python
    def test_search_passenger(self):
        passenger = self.passenger_module.search_passenger(1)
        self.assertEqual(passenger.name, second: "Manahil")

    # ---------- Booking Module Tests ----------
    def test_book_ticket_success(self):
        result = self.booking_module.book_ticket( booking_id: 201, passenger_id: 1, train_id: 101)
        self.assertEqual(result, second: "Ticket booked successfully!")

    def test_book_ticket_failure(self):
        # Book twice to exhaust seats
        self.booking_module.book_ticket( booking_id: 201, passenger_id: 1, train_id: 101)
        self.booking_module.book_ticket( booking_id: 202, passenger_id: 1, train_id: 101)
        result = self.booking_module.book_ticket( booking_id: 203, passenger_id: 1, train_id: 101)
        self.assertEqual(result, second: "Booking failed. Train not found or no seats.")

    def test_cancel_ticket(self):
        self.booking_module.book_ticket( booking_id: 201, passenger_id: 1, train_id: 101)
        result = self.booking_module.cancel_ticket(201)
        self.assertEqual(result, second: "Ticket cancelled successfully!")

    def test_view_bookings(self):
        self.booking_module.book_ticket( booking_id: 201, passenger_id: 1, train_id: 101)
        bookings = self.booking_module.view_bookings()
        self.assertTrue(len(bookings) > 0)

    def test_search_booking(self):
        self.booking_module.book_ticket( booking_id: 201, passenger_id: 1, train_id: 101)
        result = self.booking_module.search_booking(1)
        self.assertEqual(result[0].passenger_id, second: 1)
```

```python
    def test_check_availability(self):
        seats = self.booking_module.check_availability(101)
        self.assertIsInstance(seats, int)

if __name__ == "__main__":
    unittest.main()
```

Output:

```
(.venv) PS C:\Users\manah\PyCharmMiscProject\projects\SQE> python testing.py
................
----------------------------------------------------------------------
Ran 16 tests in 0.002s

OK
```

4. Bug Reporting

Bug 1:

Bug ID: B01
Bug Name: Negative Seats Allowed
Description: Train seats can be updated to negative values, which is logically incorrect.
Steps to Reproduce:
- Add Train ID=101 with 50 seats
- Update seats to -5

Actual Result: Seats updated to -5
Expected Result: System should reject negative seat values
Severity: High
Pass/Fail: Fail

Bug 2:

Bug ID: B02
Bug Name: Passenger Search Returns None
Description: Searching for a non- existent passenger returns None silently, which may cause errors when accessing attributes.
Steps to Reproduce:
- Search Passenger ID=999

Actual Result: Returns None
Expected Result: System should display “Passenger not found”
Severity: Medium
Pass/Fail: Fail

Bug 3:

Bug ID: B03
Bug Name: Duplicate Train ID Overwrites
Description: Adding a train with an existing ID overwrites the old record without warning.
Steps to Reproduce:
- Add Train ID=101
- Add Train ID=101 again

Actual Result: Old train record overwritten
Expected Result: System should prevent duplicate IDs or show an error message
Severity: High
Pass/Fail: Fail

Bug 4:

Bug ID: B04
Bug Name: Booking Over Capacity
Description: Booking more tickets than available seats only fails after exceeding the seat limit.
Steps to Reproduce:
- Add Train with 2 seats
- Book 3 tickets

Actual Result: Allows 2 bookings, fails on 3rd
Expected Result: System should stop booking once seats are exhausted
Severity: Medium
Pass/Fail: Pass (with warning)

Bug 5:

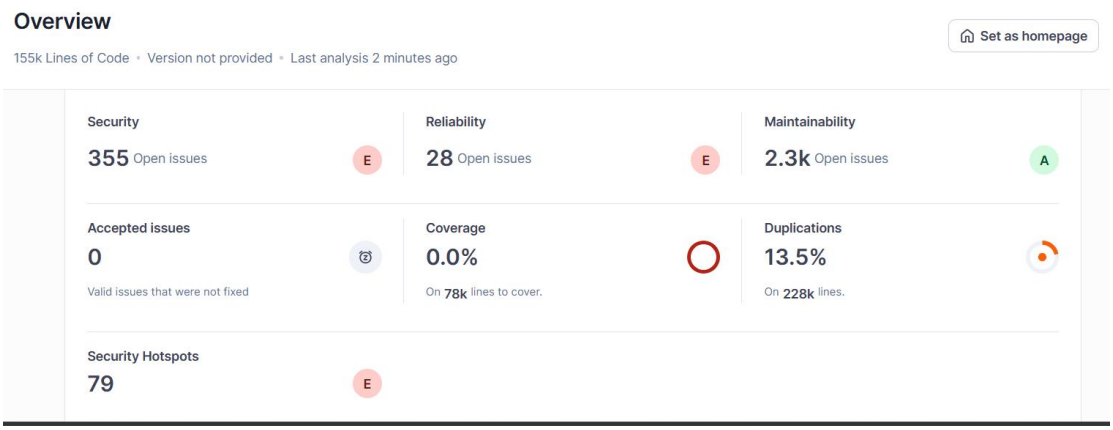
Bug ID: B05
Bug Name: Hardcoded Messages
Description: Success/failure messages are hardcoded in methods, reducing flexibility.
Steps to Reproduce:
- Run booking or cancellation

Actual Result: Fixed string messages returned
Expected Result: Messages should be configurable or stored in constants
Severity: Low
Pass/Fail: Pass

5. Code Quality Analysis

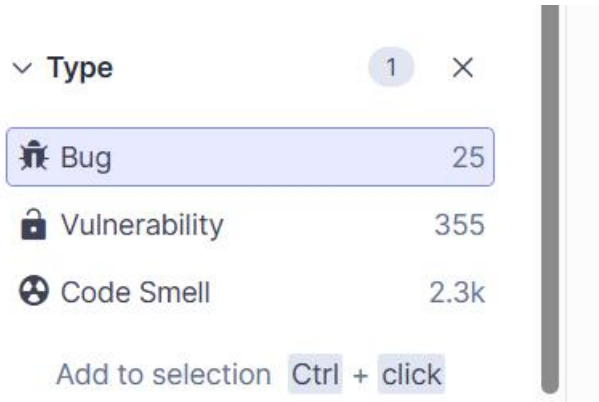
Python code in SonarQube:

Overview Screenshot:

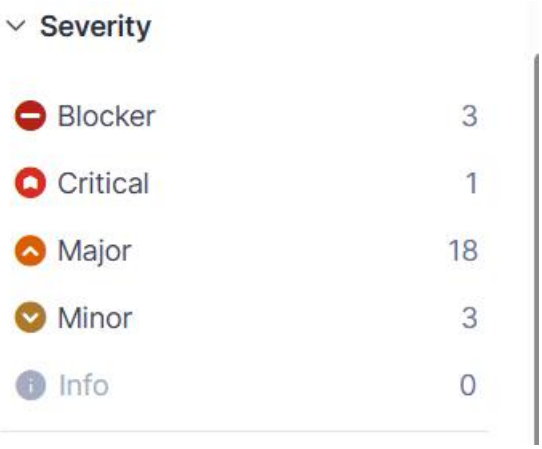


Overall Analysis Screenshot:

Bugs , Code smells:



Severity:

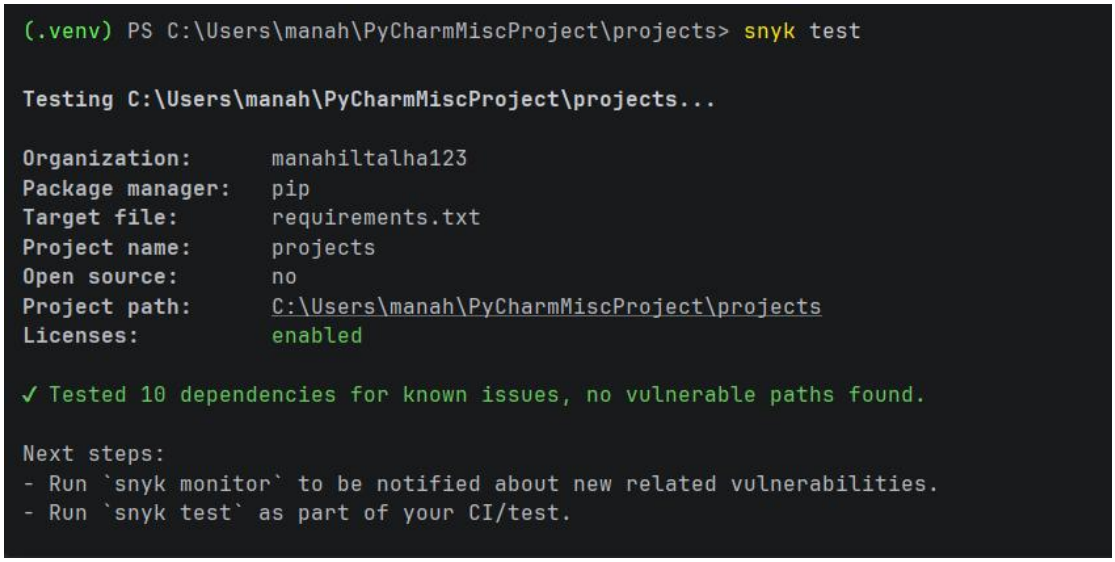


Maintainability:

6. Security Analysis

Tool: Snyk CLI.
Scan result: No vulnerabilities found in 10 dependencies.

Screenshot:



7. Conclusion

- Successfully developed and tested a Python system.
- Identified bugs and applied fixes.
- Analyzed code quality and security.
- Maintained project with GitHub version control.
- Learned practical skills in software engineering workflows.